Bidding-Based Online Policy Adaptation for Evolving Objectives

Anonymous authors
Paper under double-blind review

Keywords: Multi-Objective RL, Multi-Agent RL

Summary

The summary appears on the cover page. Although it can be identical to the abstract, it does not have to be. One might choose to omit the stated contributions in the Summary, given that they will be stated in the box below. The original abstract may also be extended to two paragraphs. The authors should ensure that the contents of the cover page fit entirely on a single page. The cover page does **not** count towards the 8–12 page limit.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Contribution(s)

1. Provide a succinct but precise list of the contribution(s) of the paper. Use contextual notes to avoid implications of contributions more significant than intended and to clarify and situate the contribution relative to prior work (see the examples below). If there is no additional context, enter "None". Try to keep each contribution to a single sentence, although multiple sentences are allowed when necessary. If using complete sentences, include punctuation. If using a single sentence fragment, you may omit the concluding period. A single contribution can be sufficient, and there is no limit on the number of contributions. Submissions will be judged mostly on the contributions claimed on their cover pages and the evidence provided to support them. Major contributions should not be claimed in the main text if they do not appear on the cover page. Overclaiming can lead to a submission being rejected, so it is important to have well-scoped contribution statements on the cover page.
   Context: None
2. The submission template for submissions to RLJ/RLC 2025
   Context: Built from previous RLC/RLJ, ICLR, and TMLR submission templates
3. *[Example of one contribution and corresponding contextual note for the paper "Policy gradient methods for reinforcement learning with function approximation" .]*
   This paper presents an expression for the policy gradient when using function approximation to represent the action-value function.
   Context: Prior work established expressions for the policy gradient without function approximation.

Bidding-Based Online Policy Adaptation for Evolving Objectives

Anonymous authors
Paper under double-blind review

Abstract

We study multi-objective reinforcement learning settings in which objectives appear or disappear at runtime. We propose a modular framework where each objective is supported by a selfish local policy, and coordination is achieved through a novel *auction*-based mechanism: policies bid for the right to execute their actions, with bids reflecting the urgency of the current state. The highest bidder selects the action, enabling a dynamic and interpretable trade-off among objectives. To succeed, each policy must not only optimize its own objective, but also reason about the presence of other goals and learn to produce calibrated bids that reflect relative priority. When objectives change, the system adapts by simply adding or removing the corresponding policies. Moreover, when objectives arise from the same parameterized family—like the class of reachability objectives parameterized by target states—identical copies of a parameterized policy can be deployed. In our implementation, the policies are trained concurrently using proximal policy optimization (PPO). We evaluate on Atari Assault and a gridworld-based path-planning task with dynamic targets. Our method achieves substantially better performance and reduced sample complexity than a single policy trained with PPO.

1 Introduction

A majority of real-world control problems require fulfilling more than one objectives simultaneously, where the objectives could be partly contradictory to each other. We consider the setting where objectives appear or disappear at deployment time, and no prior information about their arrival is available. Our running example is a coffee serving robot in an office building, where new coffee requests could appear from any place at any time, old requests could disappear even before being served, and the robot must react to updated objectives as quickly as possible. In general, any number of requests could be active at any time, and we do not assume any prior knowledge about their distribution. Our goal is to design lightweight, adaptive policies that update their behavior in light of the evolving objectives. Despite having been studied in the planning literature (**?**), this class of problems lack support in (model-free) reinforcement learning (RL), and the existing solutions from the planning setting do not readily extend.

We propose a novel policy adaptation algorithm for this class of problems. The heart of our approach is a compositional design of policies for multi-objective RL problems, where each objective is served using an independent local policy, and coordination is achieved via an online auction-based mechanism: policies bid for the right to execute their actions, with bids reflecting the urgency of the current state for fulfilling the respective objectives. The highest bidder selects the action, enabling a dynamic and interpretable trade-off among objectives. For example, imagine a situation that the coffee serving robot is approaching an intersection of two corridors, there are two active requests and two independent local policies for serving them, the first policy needs to go left, while the second policy needs to go right. Clearly a trade-off is unavoidable, and which of the policies is prioritized is decided based on auctions.

Given this compositional design, every time an objective appears or disappears, only the responsible policy needs to be added or removed, eliminating the need for modifying the whole system. Moreover, when the objectives come from the same parameterized family, like the set of all possible coffee requests parameterized by the request locations, we can design a single universal policy for this entire parameterized family, so that new additions require adding an identical copy of the policy, offer us instant adaptation. In contrast, a monolithic policy that would serve all objectives lacks this flexibility, and an attempt to train such a policy with variable number of objectives causes a dramatic degradation of training stability as well as the performance (the loss); we demonstrate this in our experiments.

We show how classical RL policies can be extended with bidding capabilities. While the obvious first step is to extend the action space with numeric bid values, there are three key challenges:

Challenge I: enforcing honest bids. We must ensure that policies bid only in proportion to their urgency, because otherwise, there is a risk of obtaining "dishonest" policies that constantly try to block others by overbidding. To circumvent this issue, during training, we require the policies to pay penalties proportional to their bid values. **KM:** I suggest, we just stick to one kind of bidding, instead of three, if in the end all-pay remains superior in all experiments. This way, it is against the interest of policies to bid too high, because otherwise, the net benefit of achieving their objectives would be lost.

Challenge II: achieving environment awareness. The bidding tactic of policies should not only depend on the current state and the own objective, but also needs to account for other objectives for maximal effectiveness. For instance, if two coffee requests appear at nearby locations, the respective policies need not bid too high to compete against the opponent. Dually, if two requests arrive from opposite directions, the bid of the more urgent policy must be high enough to counteract the opponent. Therefore, the policies must account for the objectives of opponents to bid effectively. Our approach is to model the local policy synthesis problem as an instance of multi-agent nonzerosum games, where each agent is responsible for an individual objective, and needs to learn a policy that fulfills its objective against the opponents.

Challenge III: unbounded objective count. Since the number of objectives seen during deployment cannot be predicted during training, the game-theoretic approach for environment-aware policies falls apart, because now we face a variable number of opponents for which no solution is known. We use an attention network that transforms an arbitrary number of opponent objectives to a fixed encoding, which is then fed as the input to the RL policy. Each local policy is equipped with its own attention network, which is co-trained with the actual policy for an increasing number of opponent objectives. **KM:** If we want to make the unbounded objectives a central feature, we should restrict ourselves to objectives from the same family. Otherwise, the attention pooling and the game-theoretic training approach does not work. While this is indeed a restrictive case, it is well-motivated and I do not see it as a weakness.

We implemented our framework using the proximal policy optimization (PPO) as the base learning algorithm. Using the Atari Assault and a gridworld-based path-planning task, we demonstrate the superior training stability and the performance of our policies in comparison with the baseline monolithic policies. **TODO:** Summarize the main take-away: can we give some statistics, like "the training time was 2x faster" or the "loss was 2x smaller"?

1.1 Related Work

A large body of work in multi-objective reinforcement learning (MORL) relies on *scalarization*, aggregating multiple reward functions into a single scalar objective so that standard single-objective RL algorithms can be applied. The simplest scalarization method is a weighted sum of individual rewards (Gass & Saaty, 1955), though richer nonlinear scalarization functions have also been proposed (Van Moffaert et al., 2013). A key limitation of scalarization is that the relative importance induced by the aggregation function may not align with the designer's true intent. This mismatch

can initiate a tedious debugging cycle, particularly in large-scale systems (Hayes et al., 2022). In contrast, our approach achieves a trade-off between reward components without collapsing them into a fixed scalar objective.

Other works pursue trade-offs by fixing a specific optimality criterion. Common choices include Pareto optimality (Van Moffaert & Nowé, 2014) and its approximations (Pirotta et al., 2015), as well as fairness-based criteria across reward functions (Park et al., 2024; Byeon et al., 2025; Siddique et al., 2020). These approaches typically learn a single monolithic policy that satisfies the chosen criterion. By contrast, our objective is to learn independent, selfish local policies for each reward component and compose them at runtime in a principled manner, thereby preserving modularity while still achieving a coherent global trade-off.

Relatively few works study distributed local policies for multiple rewards. A notable example is W-learning (Humphrys, 1995) and its deep RL extension (Rosero et al., 2024), where separate selfish policies are trained alongside meta-policies (W-functions) that assign each state a score reflecting its urgency. At runtime, the policy with the highest score is selected. Other approaches employ alternative aggregation mechanisms, such as ranked voting over actions (Méndez-Hernández et al., 2019), or fixed aggregation rules like summing action values across agents (Russell & Zimdars, 2003). While conceptually related, our approach is technically simpler: it relies on an engineered reward structure that enables the use of standard learning algorithms (e.g., PPO) without additional meta-policies or complex aggregation schemes. Furthermore, to the best of our knowledge, we are the first to introduce the incremental MORL setting, in which reward components can be added or removed at runtime.

The idea of bidding-based selfish policies originates from analogous techniques for multi-objective path planning problems on finite graphs (Avni et al., 2024), as well as from the broader literature on bidding games (Lazarus et al., 1999; Avni et al., 2019; 2025). These works study strategic interaction in finite arenas, where adversarial players bid for the right to determine the next move from a shared action space in pursuit of their objectives. Although these works provide strong theoretical guarantees, they do not naturally extend to infinite arenas. Moreover, players in such games are typically budget-constrained, and the central question concerns the minimum budget required to win. In contrast, we consider infinite arenas and eliminate explicit budget constraints by incorporating bidding rewards and penalties directly into the learning framework.

2 Multi-Policy Bidding for Multi-Objective RL

In this work, we consider multi-objective Markov decision processes (MO-MDP). An MO-MDP with $m \in \mathbb{N}_{>0}$ objectives is specified by a tuple $\mathcal{M} = (S, A, T, R, \mu_0)$ where S is the set of states, A is the set of actions, $T : S \times A \times S \to \mathbb{R}$ is the transition function so that $s' \in S \mapsto T(s, a, s')$ forms a probability distribution over S for all $(s, a) \in S \times A$. $\mathbf{R} = \{R_i \colon S \times A \times S \to \mathbb{R}\}_{i \in [1;m]}$ is the set of reward functions corresponding to the m objectives, and finally $\mu_0 \in \mathbb{D}(S)$ is the initial state distribution. A *policy* in an MO-MDP $\mathcal{M}$ is a function $\pi : (S \times A)^* \times S \to \mathbb{D}(A)$ that maps a history of state-action pairs and the current state to a distribution over actions. A policy naturally induces a distribution over sequences of state-action pairs that we denote $\pi_{\mathcal{M}}$. The *value* of $\pi_{\mathcal{M}}$ denoted $V(\pi_{\mathcal{M}})$ is a vector in $\mathbb{R}^m$ so that $V_i(\pi_{\mathcal{M}}) \coloneqq \mathbb{E}_{(s_i,a_i)\sim\pi_{\mathcal{M}}}[\sum_{t=0}^{\infty} \gamma^t R_i(s_i, a_i, s_{i+1})]$ where $\gamma \in (0, 1)$ is the discount factor. In the multi-objective setting, we are interested in finding a policy in the pareto-frontier of the value function which balances the tradeoffs between the objectives according to domain requirements.

We consider the compositional approach to policy synthesis for MO-MDPs, where we will design a selfish, *local* policy maximizing each individual value component, towards the fulfillment of some required global coordination requirements. The main crux is in the composition process, where each local policy may propose a different action, but the composition must decide one of the actions that will be actually executed. Importantly, the composition must be implementable in a distributed manner, meaning we will *not* use any global policy that would pick an action by analyzing all local policies and their reward functions. **TODO:** running example

We present a novel *auction*-based RL framework for compositional policy synthesis for MO-MDPs. In our framework, not only do the local policies emit actions, but also they *bid* for the privilege of executing their actions for a given number of time steps $\tau \in \mathbb{N}_{>0}$ in future. The bids are all non-negative real numbers, and the highest bidder's actions get executed for the following τ consecutive steps, with bidding ties being resolved uniformly at random. To discourage policies from always bidding high, we penalize the policies proportional to their bids. Thus, each policy can adjust its bid in proportion to the importance for it to execute its action in the current state, and the associated bidding penalty/reward aims to incentivize policies to be truthful. By making the highest bidder active, it is effectively guaranteed that the most important policy is executed. This way, we obtain a purely decentralized scheme to coordinate local policies in a given MO-MDP.

Remark 1 (On the parameter τ). The parameter τ controls how frequently the agent changes its policies. In practice, if τ is too small, the switching could be too frequent for any of the objectives to be fulfilled. For example, **TODO:** running example...

We formalize this approach by viewing the MO-MDP as an m-player general-sum Markov game (Littman, 1994) in which m players, one corresponding to each objective, compete with each other to take control of the agent and maximize their objectives. We define the game as follows:

Definition 1 (Markov bidding game)**.** Let $\mathcal{M} = (S, A, T, R, \mu_0)$ be an MO-MDP with m objectives. We define the m-player Markov bidding game as a tuple $\mathcal{B}_{\mathcal{M}} = (\hat{S}, \hat{A}, \hat{T}, P, \hat{R}, \hat{\mu}_0, \tau, \rho)$ where

- $\hat{S} = S \times [1; m] \times [0; \tau]$ is the new state space augmented with the index of a player and the number of remaining timesteps in the range $[0; \tau - 1]$ where $\tau \in \mathbb{N}_{>0}$ is the control horizon. An element of $\hat{S}$ is denoted (s, k, t).
- $\hat{A} = (A \times \mathbb{R}_{>0})^m$ represents the action space of the m players in which each player selects an action from A and a positive bid. We denote a set of actions chosen by the m players with $(\mathbf{a}, \mathbf{b}) = (a_i, b_i)_{i \in [1;m]}$.
- $\hat{T} : \hat{S} \times \hat{A} \times \hat{S} \to \mathbb{R}$ is the new transition function defined as,

$$\hat{T}((s,k,t),(\mathbf{a},\mathbf{b}),(s',k',t')) \coloneqq \begin{cases} T(s, a_k) & k = k' \wedge t' = t - 1 \\ \frac{1}{|\mathbf{b}_{\max}|} T(s, a_{k'}, s') & t = 0 \wedge t' = \tau - 1 \wedge k' \in \mathbf{b}_{\max} \\ 0 & \text{otherwise} \end{cases}$$

  where $\mathbf{b}_{\max} \coloneqq \{i \mid b_i = \max\{b_1, \ldots, b_m\}\}$ is the set of agents with maximal bids.
- $\{\hat{R}_i : \hat{S} \times \hat{A} \times \hat{S} \to \mathbb{R}\}_{i \in [1;m]}$ is the set of reward functions for the m agents with

$$\hat{R}_i((s,k,t),(\mathbf{a},\mathbf{b}),(s',k',t')) \coloneqq \begin{cases} R_i(s, a_k, s') & t > 0 \wedge i = k \\ R_i(s, a_{k'}, s') - \rho \cdot b_{k'} & t = 0 \wedge i = k' \end{cases}$$

  where $\rho \in \mathbb{R}_{>0}$ is the bid penalty factor.
- $\hat{\mu}_0 \coloneqq (\mu_0, 1, 0)$ is the initial state distribution over $\hat{S}$ induced by μ_0 and the second and third components are fixed to be $1, 0$ respectively without loss of generality.

TODO: explain the definition a bit more either before or after

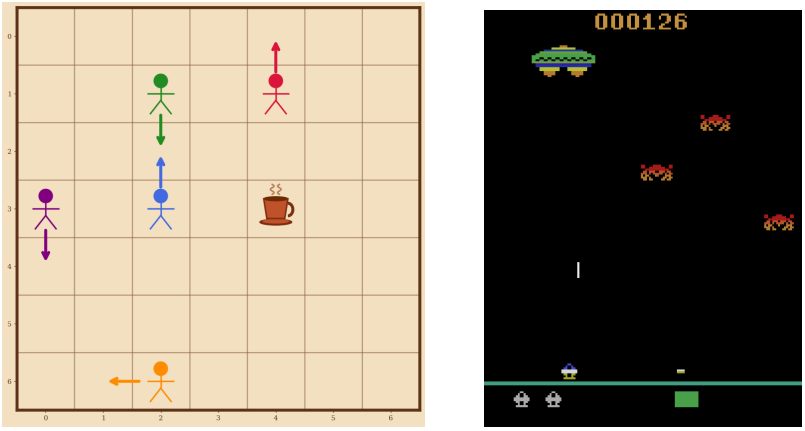
TODO: maybe have a few short comments about literature around multi-agent RL.

3 Environments with Evolving Objectives

We begin by describing the multi-objective environments that we evaluate our framework on. Despite their practical relevance, environments with multiple evolving objectives have received little attention in the multi-objective RL literature.

Coffee Shop. We implement a simple but challenging gridworld environment which consists of an $n \times n$ grid with m targets that are moving at a small rate and stochastically change directions. Each

target stays alive for a certain number of timesteps before which the agent has to reach it. Once the target expires or the agent reaches it, another new target immediately respawns at a random position in the grid. The metric the agent has to maximize is the difference between the number of targets reached and the number of targets that expired. This flexible environment also allows us to introduce more targets at deployment time.



(a) Coffee Shop (b) Atari Assault

Figure 1: Preview of the two evaluation environments.

In our experiments, we choose a 30×30 grid with 8 targets. The observations for all the local policies includes information about all the targets (positions and expiry times) and the agent; but they are reordered so that the specific target of a policy is present at the first index. This enables sharing the same actor network across policies. When the agent reaches a target before the expiration, we reward the corresponding policy and penalize it if not. Both the reward and the penalty are equal in magnitude. Furthermore, we include a distance based reward shaping for each policy with respect to its target which is crucial to the learning. **TODO:** Include this as an advantage of our framework in introduction because there is no sensible reward shaping we can do in a single-policy approach.

Atari Assault. We also test our method on the Atari game Assault in which we control a missile and shoot alien ships that move. Even though this is not seen as a multi-objective environment, we decompose it so that we can treat each alien ship as a different objective. Since these alien ships move around and there are a variable number of ships at any point of time, it is a perfect testbed for our modular approach. We check whether we can synthesize interpretable policies that split control of the missile to target different ships. However, as the standard Atari games from the Arcade Learning Environment (Bellemare et al., 2013) either use image or raw RAM observations, we cannot decompose the game score and design rewards for the multiple policies. Instead, we turn to the Object-Centric Atari environments (Delfosse et al., 2023) which exposes object positions and states by decoding RAM states. This permits us to design decomposed observations and rewards for each policy to pursue and shoot the alien ships. However, the metric we optimize is still the game score.

The observation space consists of the positions of the cannon, the alien ships and enemy missiles, current health level, and the number of remaining lives. Similar to Dynamic Targets, we reorder the positions of the alien ships in the observations for each policy. Instead of rewards based on the game score, we design the rewards for each policy. We reward a policy if it successfully destroys an alien ship and penalize it if it loses a life. **TODO:** Also talk about how each policy needs to learn to evade alien missiles.

4 Implementation

Yu et al. (2022) demonstrate that proximal policy optimization (PPO; Schulman et al. (2017)) can be effective to train multiple policies concurrently for multi-agent settings. We follow their recipe and train all the local policies with PPO. Similar to Yu et al. (2022), since the objectives in our environments share the same characteristics, we use the same actor and critic network for all the policies. At each iteration of PPO, we collect all the rollouts from each policy's perspective and sample minibatches from this pool to perform the PPO updates.

Additionally, in the Dynamic Targets environments, we employ an attention pooling architecture to encode information about all the targets. This architecture enables competition with an arbitrary number of policies (i.e., more targets) at deployment time. Each target information vector is passed through an MLP to obtain a target embedding. We then take a weighted sum of these embedding vectors where the weights are obtained by taking the dot product of the embedding vector with a learned query vector.

We implement the m-player Markov bidding game as a wrapper around the environments. This wrapper serves as an intermediary to translate the bids and actions played by the policies into actions in the environment. It also handles the action windows and bid penalties for the two bidding mechanisms. We refer to the instantiation of our method with the Winner-Pays bidding mechanism simply as Winner-Pays and similarly for All-Pay. We also consider two variants of All-Pay: All-Pay w. Local Obs and All-Pay w. No Reward Shaping. The former is the variant where each policy can only see its own target in its observation and the latter is one where we remove reward shaping in the Dynamic Targets environment.

4.1 Baselines

We compare our method against two baselines: a single policy PPO with weighted rewards and deep W-learning (DWN; Rosero et al. (2024)). For the PPO baseline, since all the objectives have the same importance across the two environments, we use the total reward across objectives at each step as the reward function. The observation space remains the same as the multi-policy setting. However, we do not implement any reward shaping for the Dynamic Targets environment due to the difficulty of specifying a meaningful dense reward signal in the presence of multiple targets. We borrow the PPO implementation from CleanRL (Huang et al., 2022). We use the specified PPO hyperparameters from CleanRL for Atari Assault and we tune the hyperparameters for the Dynamic Targets environment with 40 Optuna iterations (Akiba et al., 2019).

DWN is another multi-policy method for multi-objective RL that uses DQN for each of the local policies and additionally learns a W-network for each policy that predict scalar weights. These weights are used to choose which policy's action should be played at each step. The W-network is trained alongside the Q-network by updating it with a gradient step to predict the negative TD-error when the policy does not get to play its action. The assertion here is that the negative TD-error quantifies how important it is for the policy to play its action. The implementation is borrowed from their repository.

5 Experimental Results

We conduct experiments to show the superior performance of our method compared to the baselines. We also perform ablations with respect to all the key components of the framework; and draw insights on their importance and how we can tune them.

Learning dynamics. Figure 2 shows the learning curves for multi-policy bidding and single-policy PPO algorithms on the two environments. While the curves are more noisy with Atari Assault, they exhibit clearly distinguishable sigmoidal curves with Dynamic Targets. The curves for All-Pay and Winner-Pays methods start growing around the same time. Meanwhile, the All-Pay method with

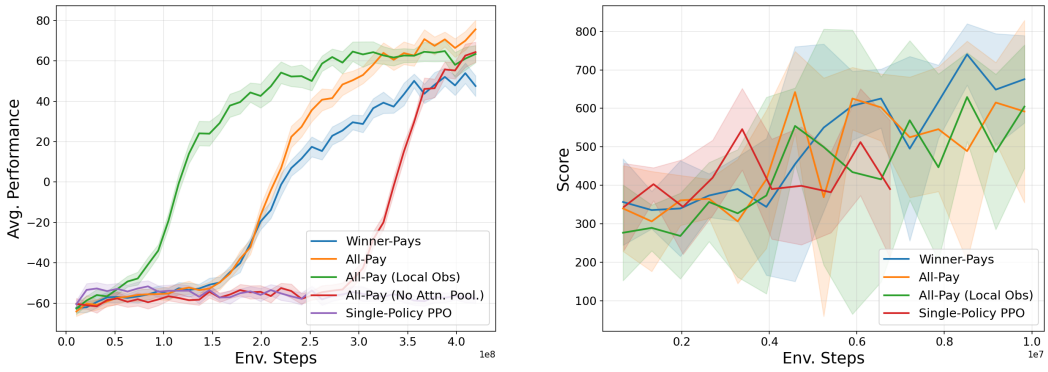


Figure 2: Learning curves for different methods on Dynamic Targets (left) and Atari Assault (right).

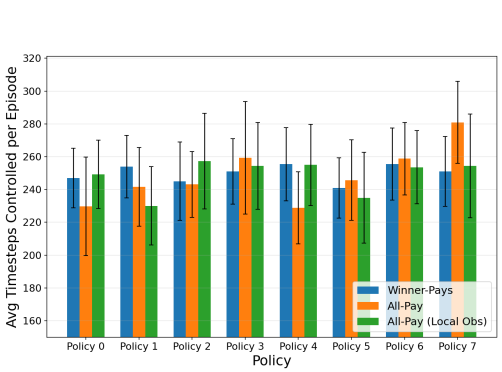
Table 1: Performance of all methods (mean with 95% CI) at the best evaluation iteration. The metric for the Dynamic Targets environment is the Performance and Score for Atari Assault.

Method	**Dynamic Targets**	**Atari Assault**
Winner-Pays	53.90 [48.84, 58.96]	740.20 [660.53, 819.87]
All-Pay	75.60 [71.12, 80.08]	642.20 [536.39, 748.01]
All-Pay w. Local Obs	64.85 [59.72, 69.98]	629.40 [513.95, 744.85]
All-Pay w. No Reward Shaping	−40.20 [−43.06, −37.34]	—
DWN (Baseline)	−41.40 [−45.48, −37.32]	294.00 [134.32, 453.68]
Single-Policy PPO (Baseline)	−51.65 [−54.50, −48.80]	546.00 [440.08, 651.92]

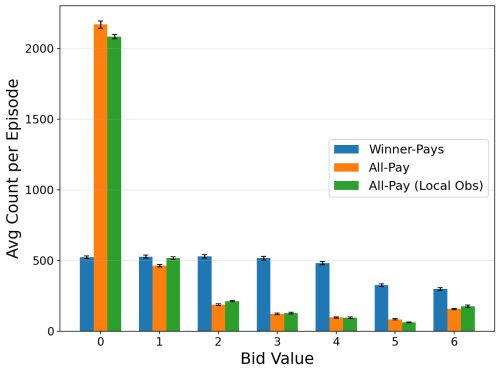
only local observations for each policy starts growing much earlier but plateaus below All-Pay. The All-Pay variant with no attention pooling for targets only starts growing much later. This validates the use of the attention pooling architecture to learn useful embeddings of other targets. We also remark that, based on the multi-agent PPO recipe of Yu et al. (2022), we also found that using a small clip coefficient ($\sim$0.05) and large batch sizes was crucial to obtain slower but stable convergence of the policies towards a joint-equilibrium.

Comparison with baselines Table 1 lists the performance of our method as well as the baselines on both environments. Our bidding-based multi-policy approaches outperform the baselines by a big margin. Across both the environments, we speculate that the poor performance of the single-policy PPO learner is due to the difficulty of credit assignment over a rollout in which there are multiple objectives that provide a positive reward. In the Dynamic Targets environment, we observe that the All-Pay multi-policy method with no reward shaping performs poorly. This indicates the difficulty of learning to navigate in the large state space with only a sparse reward signal. In the Atari Assault environment, there is a delay between shooting a missile until it reaches the alien ship and destroys it. Since we only supply information from one frame in the observation, the learner could find it hard to associate a firing of a missile and the reward from the death of a ship, especially if there is subsequent firing. On the other hand, the credit assignment becomes easier when we separate the objectives and each learner only has to focus on one target.

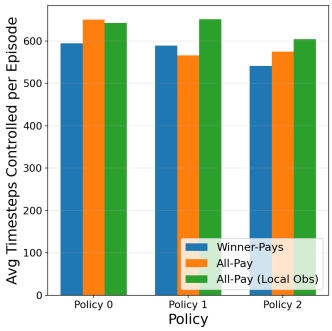
Next, let us try to reason about the poor performance of the DWN baseline. We recall that this method learns a W-network trained to predict the negative TD-error for each policy when the policy does not play its action. However, in our environments, we have a long horizon and all the objectives are symmetric with similar Q-values. This means that the TD-error is small and does not provide a strong signal to choose between policies at most timesteps. In contrast, our approach allows the policies themselves to indicate the importance of playing an action.



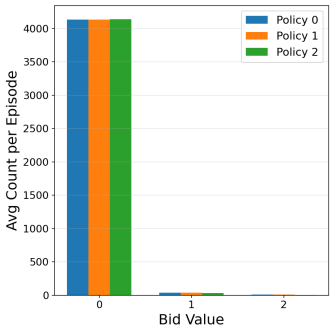
(a) Dynamic Targets: Control distribution



(b) Dynamic Targets: Bid distribution



(c) Atari Assault: Control distribution



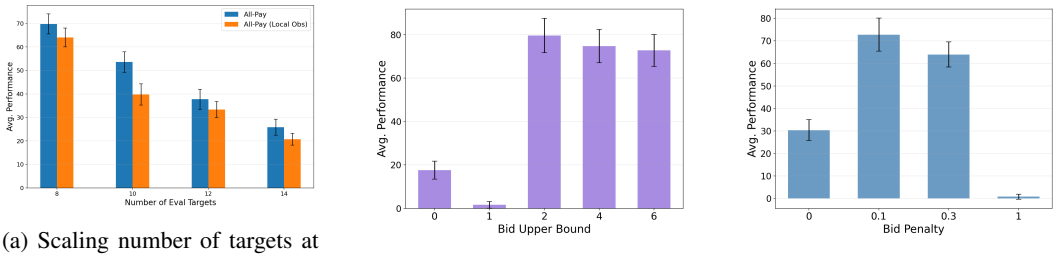
(d) Atari Assault: Bid Distribution (Winner-Pays)

Figure 3: Comparison of control and bid distributions across environments and methods. The control distribution is the number of timesteps controller by each policy, while the bid distribution is the distribution over bids by the policies.

Interpretability of policies and bidding behaviors. Video rollouts of the bidding-based policies in both environments, included in the supplementary material, demonstrate that the policies bid to take control of the agent when they can accumulate rewards in the action window. We can also observe that each policy only seeks to fulfill its own objectives, which is what we desire. Figure 3 shows the distribution of the number of timesteps that each policy was in control and the distribution over bids placed by the policies. We observe that there is a good spread of control amongst all the agents. With respect to the bid distribution, we observe different behaviors in the two environments. In the Atari Assault environment, all the policies bid 0 most of the time indicating that the agents are able to maximize their rewards through random assignment of control amongst the agents and do not need to place bids. This also intuitively makes sense as the alien ships do not automatically expire unlike the targets in Dynamic Targets. In contrast, with the Dynamic Targets environment, there is a wider spread of bids between 0 and 6. Here, we see that the All-Pay mechanism produces more conservative policies that tend to bid 0 most of the time with decreasing number of bids between 1-6. Meanwhile, the Winner-Pays mechanism has an even spread across all the possible bids indicating that the policies are willing to take risks and bid higher more often.

Scaling number of targets. We conduct an experiment with the Dynamic Targets environment in which we scale the number of targets at deployment time, i.e., we create more copies of the policy to compete with each other. Figure 4a shows that the policies continue to perform well up to 14 targets even though they were only trained to compete with up to 7 other policies. The attention pooling

mechanism plays a crucial role here as it can flexibly generate useful embeddings of an arbitrary number of competing targets.



(a) Scaling number of targets at deployment time.

(b) Ablation: Bid upper bound

(c) Ablation: Bid penalty

Figure 4: Scaling number of targets and ablations on the Dynamic Targets environment.

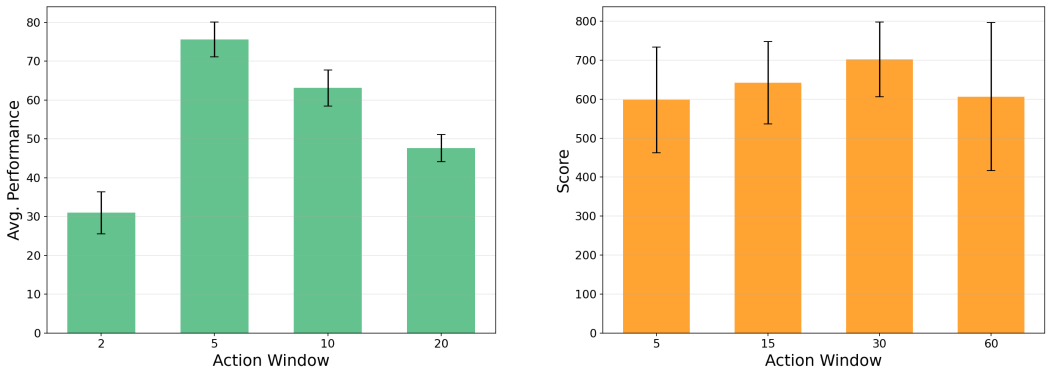


Figure 5: Effect of action window on performance: Dynamic Targets (left) and Atari Assault (right).

Further ablations. Figure 4b plots the performance of policies trained with different ranges of bids. We see that we need at least 2 levels of bids to achieve good performance. Up to just 1 bid level, the policies are unable to effectively compete with each other. Similarly, Figure 4c tries different values for the bid penalty factor. Without any penalty, the performance is low since all the policies always bid the highest trying to take control. However, if it is too big, then the policies are not willing to take the risk of placing the bid. Finally, Figure 5 compares policies trained with varying sizes of action windows. In both environments, we observe an ideal size maximizing the performance. When it is lower than this value, the control is switching between policies too fast before they are able to reach their targets. On the other hand, if it is too high, then there is not enough switching between policies to maximize all the objectives.

6 Limitations, Conclusion, and Future Directions

References

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631, 2019.

Guy Avni, Thomas A Henzinger, and Ventsislav Chonev. Infinite-duration bidding games. *Journal of the ACM (JACM)*, 66(4):1–29, 2019.

Guy Avni, Kaushik Mallik, and Suman Sadhukhan. Auction-based scheduling. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pp. 153–172. Springer, 2024.

Guy Avni, Martin Kurečka, Kaushik Mallik, Petr Novotnỳ, and Suman Sadhukhan. Bidding games on markov decision processes with quantitative reachability objectives. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, pp. 161–169, 2025.

M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, jun 2013.

Woohyeon Byeon, Giseung Park, Jongseong Chae, Amir Leshem, and Youngchul Sung. Multi-objective reinforcement learning with max-min criterion: A game-theoretic approach. *arXiv preprint arXiv:2510.20235*, 2025.

Quentin Delfosse, Jannis Blüml, Bjarne Gregori, Sebastian Sztwiertnia, and Kristian Kersting. Ocatari: Object-centric atari 2600 reinforcement learning environments, 2023.

Saul Gass and Thomas Saaty. The computational algorithm for the parametric objective function. *Naval research logistics quarterly*, 2(1-2):39–45, 1955.

Conor F Hayes, Roxana Rădulescu, Eugenio Bargiacchi, Johan Källström, Matthew Macfarlane, Mathieu Reymond, Timothy Verstraeten, Luisa M Zintgraf, Richard Dazeley, Fredrik Heintz, et al. A practical guide to multi-objective reinforcement learning and planning: Cf hayes et al. *Autonomous Agents and Multi-Agent Systems*, 36(1):26, 2022.

Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and JoÃĢo GM AraÃšjo. Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.

Mark Humphrys. W-learning: Competition among selfish q-learners. 1995.

Andrew J Lazarus, Daniel E Loeb, James G Propp, Walter R Stromquist, and Daniel H Ullman. Combinatorial games under auction play. *Games and Economic Behavior*, 27(2):229–264, 1999.

Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*, pp. 157–163. Elsevier, 1994.

Beatriz M Méndez-Hernández, Erick D Rodríguez-Bazan, Yailen Martinez-Jimenez, Pieter Libin, and Ann Nowé. A multi-objective reinforcement learning algorithm for jssp. In *International Conference on Artificial Neural Networks*, pp. 567–584. Springer, 2019.

Giseung Park, Woohyeon Byeon, Seongmin Kim, Elad Havakuk, Amir Leshem, and Youngchul Sung. The max-min formulation of multi-objective reinforcement learning: From theory to a model-free algorithm. *arXiv preprint arXiv:2406.07826*, 2024.

Matteo Pirotta, Simone Parisi, and Marcello Restelli. Multi-objective reinforcement learning with continuous pareto frontier approximation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 29, 2015.

Juan C Rosero, Nicolás Cardozo, and Ivana Dusparic. Multi-objective deep reinforcement learning optimisation in autonomous systems. In *2024 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, pp. 97–102. IEEE, 2024.

Stuart J Russell and Andrew Zimdars. Q-decomposition for reinforcement learning agents. In *Proceedings of the 20th international conference on machine learning (ICML-03)*, pp. 656–663, 2003.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Umer Siddique, Paul Weng, and Matthieu Zimmer. Learning fair policies in multi-objective (deep) reinforcement learning with average and discounted rewards. In *International Conference on Machine Learning*, pp. 8905–8915. PMLR, 2020.

Kristof Van Moffaert and Ann Nowé. Multi-objective reinforcement learning using sets of pareto dominating policies. *The Journal of Machine Learning Research*, 15(1):3483–3512, 2014.

Kristof Van Moffaert, Madalina M Drugan, and Ann Nowé. Scalarized multi-objective reinforcement learning: Novel design techniques. In *2013 IEEE symposium on adaptive dynamic programming and reinforcement learning (ADPRL)*, pp. 191–199. IEEE, 2013.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in neural information processing systems*, 35:24611–24624, 2022.